

Quantum Fractalos OS Spatial — recherche approfondie et prompt Codex pour la prochaine évolution

Orientation scientifique recommandée

La direction que tu proposes est cohérente avec ce que les campagnes 0001 → 0011 ont progressivement établi : le projet a désormais beaucoup moins besoin d'un moteur qui « cherche une anomalie » que d'un moteur qui **représente, compare et teste des structures cohérentes entre plusieurs espaces scientifiques**.

Je ferais donc évoluer Quantum Forge selon le principe suivant :

anomaly-first → coherence-first

sans supprimer les mécanismes de contrôle statistique qui ont justement rendu campaign_0011 crédible. Les détecteurs d'anomalies doivent devenir des **garde-fous passifs** et non l'objectif du moteur.

Le nouvel objectif serait :

observer des structures, transformations, invariants et relations qui persistent entre domaines

avec une distinction stricte entre :

- loi scientifique connue ;
- conséquence mathématique dérivée d'une loi ;
- analogie structurelle entre domaines ;
- relation numérique observée ;
- candidat issu d'une recherche symbolique ;
- hypothèse nouvelle non validée.

Cette distinction est essentielle. Des méthodes comme SINDy peuvent retrouver des équations gouvernantes parcimonieuses à partir de trajectoires, tandis qu'AI Feynman exploite notamment symétries, séparabilité et structure compositionnelle pour faire de la régression symbolique ; ce sont de très bonnes briques pour **chercher de la cohérence mathématique**, mais leurs sorties ne deviennent pas automatiquement des lois physiques. ¹

Le moteur pourrait donc rechercher prioritairement des correspondances du type :

même invariant, même forme différentielle, même symétrie, même loi d'échelle,
même structure topologique, même groupe sans dimension, équivalence sous changement de variables,
plutôt qu'un simple :

$$\text{observed} / \text{expected} =$$

Une recherche symbolique contrainte par les dimensions physiques est particulièrement pertinente ici : des travaux récents montrent qu'intégrer explicitement l'analyse dimensionnelle à la régression symbolique réduit l'espace de recherche et empêche une partie des expressions physiquement incohérentes. ²

Le point crucial sur la « sphère de Bloch centrale »

Ton intuition d'utiliser la sphère de Bloch comme **centre conceptuel de navigation** est très bonne comme interface.

Mais il faut séparer deux sens :

sphère de Bloch physique

et

sphère de Bloch comme centre visuel du Fractalos OS.

La sphère de Bloch représente exactement l'état d'un système à deux niveaux. Pour plusieurs qubits, `plot_bloch_multivector` de Qiskit produit une sphère par qubit en utilisant les espérances locales de X, Y, Z ; ces trois nombres ne contiennent pas, à eux seuls, toutes les corrélations et l'intrication d'un état multiqubit. ³

Plus généralement, une représentation de Bloch d'un système de dimension d nécessite naturellement un vecteur dans :

$$\mathbb{R}^{d^2-1}.$$

Pour $d > 2$, il ne faut donc pas faire croire qu'une simple boule 3D est la totalité de l'espace d'état. ⁴

La bonne architecture est donc :

la sphère centrale est une lentille de navigation et, lorsque le contexte sélectionné est réellement un qubit, elle devient en plus une représentation quantique exacte.

Pour des systèmes plus grands, elle peut être accompagnée d'une Q-sphere, d'une matrice de densité, d'un tenseur de corrélations de Pauli ou de projections choisies. Qiskit fournit notamment une Q-sphere où la taille des points encode la probabilité et leur représentation contient l'information de phase. ⁵

Modernisation de Qiskit

Au 13 août 2026, la documentation IBM expose **Qiskit SDK 2.5** comme version courante de la branche principale ; `2.5.0` est une release fonctionnelle et Qiskit 2.5 exige désormais NumPy 2.0 ou plus récent. Il faut donc demander à Codex de migrer prudemment le projet vers Qiskit 2.5.x, après inspection de toutes les dépendances et exécution de la suite de tests. ⁶

L'évolution la plus importante pour Quantum Fractalos n'est pas d'ajouter des dizaines de nouveaux algorithmes quantiques, mais de **nettoyer la couche d'abstraction Qiskit**.

Les anciennes primitives V1 ont largement été remplacées par les interfaces V2. Qiskit fournit `StatevectorSampler` et `StatevectorEstimator` comme implémentations de référence V2 ; l'API V2 est notamment conçue pour traiter efficacement des sweeps de paramètres et d'observables. ⁷

Je recommanderais cette séparation :

Besoin Fractalos	Brique Qiskit
état pur exact	<code>Statevector</code>
état mixte	<code>DensityMatrix</code>
shots simulés	<code>StatevectorSampler</code>
espérances	<code>StatevectorEstimator</code>
opérateurs de Pauli	<code>SparsePauliOp</code>
observables structurées	<code>SparseObservable</code>
paramètres contrôlés par espaces externes	<code>Parameter</code> / <code>ParameterVector</code>
persistance exacte des circuits	QPY
affichage de référence	visualisations Qiskit
affichage interactif Fractalos	moteur 3D séparé

`StatevectorSampler` est une primitive V2 construite sur la simulation par vecteur d'état et expose un seed reproductible ; sa limitation importante est qu'elle ne traite pas les mesures intermédiaires de circuit comme le ferait une implémentation adaptée aux circuits dynamiques. ⁸

`StatevectorEstimator` utilise également la simulation complète du vecteur d'état, prend en charge les PUBs vectorisés et reste actuellement limité aux observables basées sur les Pauli dans cette implémentation. ⁹

`Statevector` doit rester disponible directement pour le moteur géométrique : Qiskit sait construire un état à partir d'un circuit, le faire évoluer et calculer des valeurs d'espérance. ¹⁰

`DensityMatrix` devient important dès que Fractalos souhaite représenter explicitement des états mixtes ou certaines transformations ouvertes plutôt que de tout ramener à un état pur. ¹¹

Pour les observables, Qiskit possède maintenant `SparseObservable`, qui représente des sommes d'opérateurs de Pauli et de projecteurs de Pauli sous forme creuse ; `SparsePauliOp` demeure également une représentation naturelle des opérateurs multiqubits de Pauli. ¹²

Pour sauvegarder les circuits qui sous-tendent une scène conceptuelle, QPY est préférable à un format ad hoc : Qiskit décrit QPY comme un format binaire conçu pour conserver la structure des `QuantumCircuit`, être indépendant de la version Python et permettre une compatibilité ascendante, alors que QASM correspond à un autre niveau d'abstraction et peut perdre certains éléments spécifiques aux objets Qiskit. ¹³

Il existe également une motivation pratique à cibler au moins Qiskit 2.5 : cette release corrige notamment un problème où `Statevector.evolve()` pouvait modifier l'état d'entrée lorsqu'une instruction comportait une phase globale. ¹⁴

Ce qu'il ne faut pas faire

Je ne demanderais pas à Codex de transformer toute notion « fractale » en circuit quantique.

Le système spatial doit être :

mathématique + géométrie + scientifique

avec Qiskit comme **l'un des moteurs de calcul**, pas comme ontologie universelle.

Autrement, on risquerait de forcer des concepts classiques, temporels, thermodynamiques ou géométriques dans des qubits sans justification scientifique.

Architecture de Quantum Fractalos OS Spatial

Le concept le plus puissant de ta demande peut être formalisé avec **des espaces imbriqués reliés par des fonctions**.

Au lieu d'avoir un seul monde XYZ :

$$\mathcal{S}_1 = (X, Y, Z),$$

on introduit plusieurs espaces :

$$\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_n$$

et des applications :

$$F_{ij} : \mathcal{S}_i \rightarrow \mathcal{S}_j.$$

Ton exemple devient alors extrêmement propre.

On définit :

$$\mathcal{S}_{\text{world}} = (X, Y, Z)$$

et :

$$\mathcal{S}_{\text{radius}} = (Y_1).$$

Puis :

$$F(Y_1) = R.$$

La sphère dans XYZ satisfait :

$$x^2 + y^2 + z^2 = R^2.$$

Donc :

$$Y_1 \longrightarrow R \longrightarrow \text{géométrie dans XYZ.}$$

Le slider n'est pas « une quatrième dimension physique » par magie. Il est une coordonnée d'un **espace de paramètres** couplé à l'espace XYZ.

C'est justement ce mécanisme qui permet de faire coexister autant d'espaces que nécessaire sans confondre leur signification.

Le modèle de données central

Je construirais le cœur du système autour de cinq types.

```
ConceptSpace
Frame / Envelope
Mapping
Action
ScientificLaw
```

Un `ConceptSpace` pourrait posséder :

```
id
name
dimension
axes
units
coordinate_system
parent_space
children
depth
scale
geometry
state
time_model
metadata
```

Un `Mapping` :

$$F : A \rightarrow B$$

avec :

```
source_space
target_space
input_axes
output_axes
expression
```

```
units
domain
codomain
invertibility
assumptions
```

Un **Action** :

```
origin
target
direction
scope
operator
magnitude
time_window
propagation
```

Et **scope** pourrait prendre :

```
INTERIOR
BOUNDARY
EXTERIOR
BULK
ALL
```

tandis que **direction** serait :

```
INWARD
OUTWARD
BIDIRECTIONAL
LOCAL
```

Cela formalise exactement ton idée :

agir sur un cadre depuis un englobant plus petit ou plus grand, puis laisser l'action se propager vers l'intérieur ou l'extérieur.

Le « fractal » comme profondeur conceptuelle

Je n'utiliserais pas ici « fractal » pour prétendre que l'Univers possède nécessairement une géométrie fractale.

Je lui donnerais une définition logicielle précise :

Fractal Depth = hiérarchie récursive d'espaces et d'échelles

Par exemple :

depth -3	structure sublocale
depth -2	composant
depth -1	système
depth 0	contexte actuellement étudié
depth +1	environnement
depth +2	domaine
depth +3	métadomaine

L'utilisateur peut déplacer le `depth_origin`.

Ainsi n'importe quel niveau peut devenir le nouveau :

$$n = 0.$$

La sphère centrale qui rétrécit lorsque l'espace conceptuel grandit

Ton idée peut être implémentée exactement, mais comme **fonction de représentation**.

Par exemple :

$$R_{\text{Bloch}}(z) = R_0 e^{-\alpha z},$$

où z est la profondeur de zoom conceptuel vers les grandes échelles.

Ou :

$$R_{\text{Bloch}}(s) = \frac{R_0}{(1+s)^\alpha}.$$

Donc plus le contexte représenté devient grand :

$$s \uparrow \Rightarrow R_{\text{Bloch}} \downarrow.$$

Et inversement :

$$s \downarrow \Rightarrow R_{\text{Bloch}} \uparrow.$$

Ce rayon visuel ne doit jamais modifier le vecteur de Bloch physique.

Il faut maintenir séparément :

```
physical_bloch_vector
display_radius
conceptual_scale
camera_scale
```

C'est une séparation extrêmement importante.

Le quadrillage spatial à profondeur arbitraire

Le quadrillage peut être logarithmique.

Pour un facteur :

$$b = 10$$

les niveaux sont :

$$\dots, 10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3, \dots$$

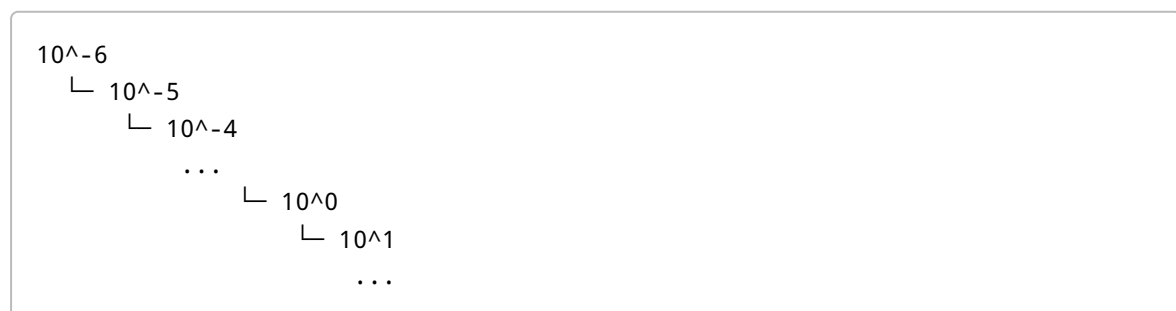
ou, pour certaines structures numériques :

$$b = 2.$$

Le moteur ne doit afficher que les niveaux utiles au zoom courant, mais conserver une hiérarchie théorique arbitraire :

$$n \in \mathbb{Z}.$$

Visuellement :



La scène devient donc une sorte de **microscope / télescope conceptuel**.

Pour une implémentation Web, Three.js fournit déjà les primitives nécessaires à une caméra 3D interactive, au zoom/pan/orbite via `OrbitControls`, au picking des objets via raycasting et à l'organisation des objets visibles en couches. ¹⁵

Représentation des dimensions une à cinq

Le principe le plus rigoureux est :

Dimension conceptuelle	Affichage
1D	axe / slider / courbe
2D	plan / carte / champ
3D	géométrie directe
4D	tranche ou trajectoire 3D commandée par une quatrième coordonnée
5D	3D visible + deux coordonnées externes

Dimension conceptuelle	Affichage
N-D	projection/slice + contrôleurs des dimensions cachées

Pour :

$$(X, Y, Z, T)$$

il faut donc proposer :

$$V_T(X, Y, Z)$$

c'est-à-dire une scène XYZ évaluée à T .

Le slider temporel agit sur :

$$T.$$

Une autre représentation peut afficher simultanément la trajectoire :

$$(X(T), Y(T), Z(T)).$$

Ainsi l'utilisateur voit une scène 3D mais **navigue réellement dans un objet à quatre variables**.

Pour cinq dimensions :

$$(X, Y, Z, T, \lambda)$$

on peut avoir :

```
vue principale : XYZ
slider 1 : T
slider 2 : λ
```

et :

$$Scene = F(X, Y, Z \mid T, \lambda).$$

Ce système généralise naturellement à davantage de dimensions.

Il faut simplement éviter de dire qu'un écran affiche « directement » un espace 5D : il en affiche une **projection, une tranche ou un encodage**.

Moteur de cohérence entre sciences

C'est probablement l'évolution la plus intéressante à long terme.

Je créerais un nouveau composant :

CoherenceEngine

qui remplace comme objectif principal :

AnomalySearchEngine

sans supprimer le second.

Le moteur devrait examiner chaque paire ou groupe de lois sous plusieurs angles.

Cohérence dimensionnelle

Avant toute comparaison :

$$[LHS] = [RHS].$$

Une équation qui échoue à ce test est immédiatement marquée incohérente pour le système d'unités considéré.

Cohérence algébrique

Deux lois apparemment différentes peuvent devenir identiques après :

- changement de variables ;
- normalisation ;
- passage aux grandeurs sans dimension ;
- translation ;
- rotation ;
- changement de coordonnées ;
- composition de fonctions.

Le moteur doit rechercher :

$$F_1 \sim T^{-1} \circ F_2 \circ T.$$

Invariants

Chercher ce qui reste identique lorsque le système évolue :

$$I(x(t)) = \text{constante}.$$

L'identification automatisée de structures gouvernantes ou invariantes à partir de trajectoires est précisément un axe de recherche actif ; SINDy recherche des modèles dynamiques parcimonieux dans une bibliothèque de fonctions candidates. ¹⁶

Symétries

Tester si :

$$F(Tx) = TF(x)$$

ou :

$$F(Tx) = F(x).$$

Plutôt que demander :

« qu'est-ce qui sort de la tolérance ? »

le moteur demande :

« quelle transformation laisse cette structure stable ? »

Lois d'échelle

Chercher :

$$Y \propto X^\alpha.$$

Puis comparer α , les grandeurs sans dimension et les limites asymptotiques entre domaines.

Cohérence topologique

Comparer :

- connexité ;
- nombre de composantes ;
- cycles ;
- chemins ;
- invariants de graphe ;
- transformations préservant certaines structures.

C'est particulièrement cohérent avec l'évolution de `topology_graph` dans les campagnes récentes.

Cohérence dynamique

Comparer :

$$\dot{x} = F(x)$$

entre systèmes en cherchant :

- mêmes points fixes ;
- mêmes types de stabilité ;
- mêmes attracteurs ;
- mêmes conservations ;
- mêmes bifurcations ;
- mêmes équations après reparamétrisation.

Régression symbolique sous contrainte

Les méthodes de découverte symbolique sont pertinentes comme **générateurs de candidats**, pas comme oracle de vérité. AI Feynman exploite notamment la structure mathématique et physique pour simplifier la recherche, tandis que SINDy cherche des systèmes dynamiques parcimonieux à partir de données. ¹⁷

Le cycle devrait être :

generate → unit check → symmetry check → holdout → cross-domain comparison → candidate.

Jamais :

fit data → new law.

Graphe de cohérence

Chaque loi devient un nœud.

Les relations peuvent être :

```
SAME_FORM
TRANSFORM_EQUIVALENT
SHARED_INVARIANT
SHARED_SYMMETRY
SCALE_RELATED
DIMENSIONLESS_ANALOGUE
LIMIT_OF
GENERALIZES
SPECIAL_CASE_OF
PARAMETER_CONTROLS
OBSERVATION_SUPPORTS
CONTRADICTS
```

Dans Fractalos OS, ces relations deviennent visuelles.

Un utilisateur pourrait donc regarder :

```
mécanique
  ↘
   invariant
  ↗
contrôle
```

ou :

```
thermodynamique
  \ scaling law ✓
théorie des réseaux
```

sans prétendre que deux phénomènes ayant la même équation sont nécessairement le même phénomène physique.

Prompt Codex prêt à utiliser

Le prompt ci-dessous est volontairement très prescriptif. Il demande d'abord à Codex d'inspecter le dépôt, afin qu'il adapte l'architecture aux fichiers réellement présents au lieu d'inventer une arborescence déconnectée du projet.

Tu travailles sur Quantum Forge / Quantum Fractalos OS.

MISSION

=====

Faire évoluer le projet vers une nouvelle couche appelée :

Quantum Fractalos OS Spatial

Le but n'est PLUS de chercher automatiquement des anomalies comme objectif principal.

Le nouveau but est :

1. représenter spatialement des domaines, cadres, lois, formules, paramètres et états scientifiques ;
2. représenter plusieurs espaces conceptuels de dimensions différentes ;
3. permettre à des fonctions de relier ces espaces ;
4. permettre des actions qui se propagent d'un espace vers un espace parent, enfant ou voisin ;
5. rechercher des points de cohérence mathématique/scientifique récurrents entre plusieurs domaines ;
6. conserver tous les mécanismes existants de validation, reproductibilité, ledger, contrôle statistique et audit.

IMPORTANT

=====

Avant toute modification :

- inspecte l'arborescence complète du repository ;
- lis pyproject.toml / requirements / package metadata ;
- trouve toutes les utilisations actuelles de Qiskit ;
- trouve Statevector, Sampler, Estimator, Backend, Aer, quantum_info, visualization, circuit serialization ;
- trouve les modules campaign/protocol/experiments/artifacts ;

- exécute les tests existants ;
- ne modifie pas silencieusement la logique scientifique historique qui a généré les campagnes précédentes ;
- préserve la reproductibilité des campagnes existantes ;
- toute modification du protocole scientifique doit avoir une nouvelle version explicite.

Ne commence à coder qu'après cette inspection.

```
=====
PARTIE A – MODERNISATION QISKIT
=====
```

Objectif cible : API Qiskit moderne 2.x, idéalement 2.5.x si l'environnement du repository le permet sans casser ses dépendances.

1. Vérifie la version Qiskit réellement installée.
2. Identifie toutes les API obsolètes.
3. Migre les primitives anciennes vers les interfaces V2 lorsque pertinent.
4. Crée une couche d'abstraction locale :

```
quantum/
  state_engine
  sampling_engine
  observable_engine
  circuit_store
  qiskit_adapter
```

Le reste de Quantum Forge ne doit pas dépendre directement de détails d'API Qiskit dispersés dans tout le code.

5. Distingue explicitement :

```
PureState
MixedState
Observable
Circuit
SampledMeasurement
ExactExpectation
```

6. Utilise les abstractions Qiskit modernes appropriées pour :

- état pur ;
- matrice de densité ;
- sampling avec seed ;
- estimation d'observables ;
- paramètres ;
- opérateurs de Pauli.

7. La reproductibilité RNG reste obligatoire.

Chaque exécution doit pouvoir exposer :

```
master_seed
execution_seed
replicate_seed
seed_domain
```

Aucun replicate déclaré indépendant ne peut partager son seed avec son parent.

8. Ajoute une vérification :

```
assert no duplicated execution seeds
```

pour les exécutions qui sont déclarées indépendantes.

9. Ajoute des tests de non-régression sur :

- état de Bell ;
- rotation Rx/Ry/Rz ;
- phase globale ;
- valeurs d'espérance X/Y/Z ;
- sampling reproductible ;
- circuits paramétrés ;
- état multiqubit ;
- matrice de densité.

10. Conserve les visualisations Qiskit comme référence/test, mais ne les utilise pas comme moteur graphique principal de Fractalos.

11. Ajoute une sérialisation standard des circuits adaptée à Qiskit, et conserve aussi un format JSON Fractalos indépendant de Qiskit pour les espaces conceptuels.

```
=====
PARTIE B – QUANTUM FRACTALOS SPATIAL CORE
=====
```

Créer un moteur générique de représentation conceptuelle.

Ne lier AUCUNE notion de "dimension conceptuelle" automatiquement à un qubit.

Créer au minimum les types :

```
ConceptSpace
ConceptAxis
Frame
Mapping
```

```
Action
Boundary
ScientificObject
ScientificLaw
ObservableBinding
QuantumBinding
```

ConceptSpace doit supporter dimension ≥ 1 .

Exemple :

```
ConceptSpace(
    id="world",
    dimension=3,
    axes=["X", "Y", "Z"]
)
```

et :

```
ConceptSpace(
    id="radius_parameter",
    dimension=1,
    axes=["Y1"]
)
```

Créer un Mapping :

```
radius_parameter.Y1 -> world.sphere.radius
```

Ainsi modifier Y1 modifie réellement la sphère rendue dans XYZ.

Le Mapping ne doit pas être codé en dur pour cet exemple :
il doit être générique.

Supporter :

```
scalar -> scalar
scalar -> vector
vector -> scalar
vector -> vector
function -> geometry
function -> state
state -> observable
parameter -> Qiskit Parameter
```

```
=====
PARTIE C – FRAMES ET PROFONDEUR FRACTALE
=====
```

Un Frame est un englobant conceptuel.

Un Frame peut contenir :

- spaces
- subframes
- laws
- objects
- mappings
- actions

Créer une profondeur récursive entière :

depth $\in \mathbb{Z}$

Le système doit permettre de choisir n'importe quel Frame comme depth=0.

Implémenter une échelle récursive / logarithmique :

scale(depth) = base ** depth

avec base configurable.

Supporter au minimum base=10 et base=2.

Créer un quadrillage 3D multi-échelle.

Les graduations doivent pouvoir représenter :

$10^{-n} \dots 10^0 \dots 10^n$

sans avoir à instancier toutes les grilles en mémoire.

Utiliser un système de level-of-detail.

Afficher uniquement les niveaux utiles au zoom courant.

=====
PARTIE D – SPHÈRE DE BLOCH CENTRALE
=====

Créer une vraie représentation 3D interactive de la sphère de Bloch.

Elle doit comporter :

- X
- Y
- Z
- équateur
- méridiens
- vecteur d'état
- coordonnées
- labels

transparence
grille

La donnée physique et la taille visuelle DOIVENT être séparées :

physical_bloch_vector
conceptual_display_radius

Quand l'utilisateur élargit le contexte conceptuel :

conceptual scale ↑
displayed Bloch sphere radius ↓

Quand il plonge vers une profondeur plus locale :

conceptual scale ↓
displayed Bloch sphere radius ↑

Cette transformation est VISUELLE.

Elle ne doit jamais modifier l'état quantique physique.

Créer une fonction monotone configurable, par exemple :

$$\text{display_radius} = r_0 * \exp(-\alpha * \text{conceptual_zoom})$$

avec bornes min/max.

Pour un qubit réel, les coordonnées visibles doivent venir du vrai état quantique.

Pour un système multiqubit :

- ne jamais prétendre qu'une seule sphère 3D représente l'état complet ;
- permettre la sélection d'un qubit ;
- afficher son état local ;
- ajouter un panneau de corrélations / observables multiqubit ;
- prévoir une vue alternative de l'état complet.

=====
PARTIE E – RENDU 1D / 2D / 3D / 4D / 5D
=====

Créer un système de vues dimensionnelles.

1D

Représentation :

axis

scalar
curve
slider

Exemples :

time
radius
temperature
coupling
probability

2D

Représentation :

XY plane
parameter plane
vector field
heatmap
contour

3D

Représentation géométrique directe :

XYZ
points
lines
surfaces
volumes
vector fields

4D

Ne pas prétendre afficher directement quatre dimensions spatiales complètes.

Supporter au minimum :

XYZ + T

La vue principale affiche XYZ.

T est commandé par :

slider
timeline
playback

Une modification de T recalcule la tranche XYZ correspondante.

Supporter également :

- trajectory
- persistence trail
- temporal ghosting

5D

Supporter au minimum :

XYZ + P1 + P2

XYZ est visible.

P1 et P2 sont pilotables indépendamment.

Exemple :

- XYZ = world
- P1 = time
- P2 = coupling

Le moteur recalcule la géométrie / état / observables lorsque P1 ou P2 change.

N-D

Concevoir les structures pour N dimensions.

Au-delà de trois dimensions visibles, utiliser :

- slicing
- projection
- parameter controls
- small multiples
- animation

La représentation doit toujours indiquer quelles dimensions sont :

- visible
- projected
- fixed
- controlled

=====

PARTIE F – ACTIONS ENTRE ESPACES

=====

Créer un moteur Action.

Une action possède :

- source_space
- target_space
- operator
- magnitude
- direction
- scope
- propagation
- time_window

direction :

- LOCAL
- INWARD
- OUTWARD
- BIDIRECTIONAL

scope :

- INTERIOR
- BOUNDARY
- EXTERIOR
- BULK
- ALL

Exemple :

Une action provenant d'un espace parent peut être appliquée uniquement à la frontière extérieure d'un enfant.

Une action provenant d'un enfant peut être propagée vers son parent.

Créer un graphe explicite de propagation.

Protéger contre :

- cycles infinis
- réursion non bornée
- unités incompatibles
- dimensions incompatibles

Chaque action doit produire une trace :

- action_id
- origin
- targets

```
transformations
before_state
after_state
```

```
=====
PARTIE G – SCIENTIFIC LAW REGISTRY
=====
```

Créer ScientificLaw.

Champs minimum :

```
id
name
domain
equation
variables
units
assumptions
validity_domain
boundary_conditions
symmetries
invariants
parameters
evidence_status
provenance
```

evidence_status :

```
ESTABLISHED
EMPIRICAL
DERIVED
MODEL_DEFINED
CANDIDATE
SPECULATIVE
```

Ne jamais afficher une formule générée automatiquement comme ESTABLISHED.

Créer une bibliothèque initiale minuscule mais bien testée,
pas une encyclopédie incontrôlée.

Utiliser d'abord des systèmes de démonstration simples :

```
harmonic oscillator
damped oscillator
diffusion
logistic dynamics
simple Hamiltonian
Bloch rotation
```

```
=====
```

PARTIE H – COHERENCE ENGINE

=====

Le nouveau moteur par défaut doit être :

CoherenceEngine

Il remplace Anomaly Search comme OBJECTIF par défaut.

Ne supprime pas les contrôles d'anomalies historiques.
Ils restent disponibles comme diagnostics et garde-fous.

Le CoherenceEngine analyse :

1. dimensional consistency
2. algebraic equivalence
3. coordinate-transform equivalence
4. shared invariants
5. shared symmetries
6. scale relations
7. topology
8. boundary behavior
9. dynamic behavior
10. empirical holdout agreement

Créer des relations :

SAME_FORM
TRANSFORM_EQUIVALENT
SHARED_INVARIANT
SHARED_SYMMETRY
SCALE_RELATED
DIMENSIONLESS_ANALOGUE
SPECIAL_CASE_OF
GENERALIZES
LIMIT_OF
PARAMETER_CONTROLS
OBSERVATION_SUPPORTS
CONTRADICTS

Ne pas réduire immédiatement toutes ces informations à un score unique.

Conserver un vecteur de cohérence :

dimensional
algebraic
symmetry
invariant
scale
topology
empirical

et calculer éventuellement un résumé secondaire.

Une correspondance interdisciplinaire doit rester une :

COHERENCE_CANDIDATE

tant qu'elle n'a pas été validée.

```
=====
PARTIE I – SYMBOLIC / LAW EXPLORATION
=====
```

Créer une API extensible pour un futur moteur de symbolic regression.

Pour aujourd'hui, ne pas implémenter un énorme moteur ML si aucune dépendance équivalente n'existe déjà.

Créer d'abord :

```
SymbolicCandidate
SymbolicVerifier
UnitVerifier
HoldoutVerifier
```

Pipeline :

```
candidate generation
->
dimensional verification
->
numerical verification
->
invariant/symmetry tests
->
holdout
->
coherence graph
```

Les candidats doivent rester explicitement séparés des lois établies.

```
=====
PARTIE J – INTERFACE 3D
=====
```

Inspecte d'abord le frontend existant.

S'il existe déjà un renderer WebGL/WebGPU, réutilise-le.

Sinon créer un frontend minimal TypeScript + Three.js.

La vue doit permettre AUJOURD'HUI :

1. rotation caméra ;
2. zoom ;
3. pan ;
4. sélection d'objet ;
5. affichage des coordonnées ;
6. sphère de Bloch ;
7. grille XYZ ;
8. échelle fractale ;
9. slider de profondeur ;
10. slider temporel T ;
11. deux sliders génériques P1/P2 ;
12. affichage d'une sphère XYZ dont le rayon dépend de Y1 ;
13. affichage des mappings entre espaces ;
14. activation/désactivation des différents ConceptSpaces ;
15. inspecteur latéral de propriétés.

Ajouter une scène de démonstration obligatoire :

FRAME:

UniverseDemo

SPACE A:

XYZ

SPACE B:

Y1

OBJECT:

sphere

MAPPING:

radius = Y1

Quand Y1 varie :

la taille de sphere varie dans XYZ.

Ajouter ensuite :

SPACE C:

T

MAPPING:

position = trajectory(T)

Le slider T doit déplacer la sphère dans XYZ.

Ajouter :

SPACE D:
quantum_parameter

MAPPING:
quantum_parameter -> circuit rotation angle

Le changement doit :

mettre à jour le circuit ;
recalculer l'état ;
recalculer le Bloch vector ;
mettre à jour la sphère de Bloch.

=====
PARTIE K – OBSERVABILITÉ
=====

L'interface doit permettre de cliquer sur une relation et afficher :

source
target
equation
units
assumptions
status
provenance

Cliquer sur une loi doit afficher :

equation
parameters
applicable domain
mappings
observed values

Cliquer sur la sphère de Bloch doit afficher :

state
X expectation
Y expectation
Z expectation
purity if available
selected qubit

=====
PARTIE L – COMPATIBILITÉ AVEC LES CAMPAGNES
=====

NE PAS casser le pipeline historique de campagnes.

Créer un mode :

legacy_campaign_compatibility

Les anciens artefacts doivent toujours être lisibles.

Les fonctions de classification :

EXPECTED
STATISTICAL_FLUCTUATION
etc.

restent compatibles.

Mais le nouveau scheduler Fractalos utilise par défaut :

objective = COHERENCE_MAPPING

et non :

objective = ANOMALY_DISCOVERY

Une recherche d'anomalies ne doit être déclenchée que par configuration explicite.

=====
PARTIE M – PROVENANCE
=====

Corriger définitivement la reproductibilité des sources.

Ajouter au futur artefact :

SOURCE_SNAPSHOT/
environment lock
Qiskit version
NumPy version
source SHA-256
protocol version
implementation version

Ajouter :

design_fingerprint
execution_fingerprint

avec sens strict :

même expérience -> même design_fingerprint
nouvelle exécution -> nouveau execution_fingerprint

=====

PARTIE N – TESTS

=====

Ajouter des tests unitaires pour :

- ConceptSpace 1D ;
- ConceptSpace 2D ;
- ConceptSpace 3D ;
- ConceptSpace 4D ;
- ConceptSpace 5D ;
- mapping 1D -> 3D ;
- mapping parameter -> Qiskit circuit ;
- inward propagation ;
- outward propagation ;
- boundary-only action ;
- cycle prevention ;
- dimensional incompatibility ;
- Bloch visual scaling independent of physical state ;
- fresh execution seeds ;
- source snapshot ;
- coherence relations.

Ajouter des tests d'intégration pour :

- A.
 - Y1 slider -> XYZ sphere radius
- B.
 - T slider -> XYZ position
- C.
 - quantum angle slider
 - > circuit
 - > quantum state
 - > Bloch vector
 - > 3D renderer
- D.
 - zoom conceptual
 - > display radius changes
 - > physical state DOES NOT change
- E.
 - two mathematically equivalent formulas
 - > TRANSFORM_EQUIVALENT
- F.
 - dimensionally invalid formula
 - > rejected by UnitVerifier

=====

PARTIE O – LIVRABLE AUJOURD'HUI

=====

Priorité absolue : obtenir un MVP visible aujourd'hui.

Le MVP est considéré terminé lorsque :

1. Quantum Forge démarre sans casser les tests existants.
2. Qiskit passe par la nouvelle abstraction.
3. Une fenêtre/vue 3D interactive apparaît.
4. Elle contient une vraie sphère de Bloch interactive.
5. Elle contient une grille XYZ multi-échelle.
6. Le zoom conceptuel fait rétrécir la sphère de Bloch visuelle quand l'espace représenté grandit.
7. Le Bloch vector physique reste inchangé durant ce zoom.
8. Un slider Y1 modifie le rayon d'une seconde sphère dans XYZ.
9. Un slider T permet de naviguer dans une évolution XYZ(T).
10. Deux sliders permettent une démonstration 5D :
$$XYZ + T + P$$
11. Plusieurs ConceptSpaces peuvent être activés simultanément.
12. Un Mapping est visible et inspectable.
13. Une Action INWARD ou OUTWARD est démontrée.
14. Le CoherenceEngine peut identifier au moins une relation simple entre deux modèles de démonstration.
15. Aucun mécanisme ne décrit une projection 4D/5D comme une observation directe de toutes les dimensions.

=====

PARTIE P – QUALITÉ

=====

Pas de prototype jetable.

Exiger :

typing

dataclasses / models explicites
tests
documentation
séparation backend/rendering
séparation physique/visualisation
unités explicites
seeds explicites
erreurs explicites

Ne jamais mélanger :

conceptual_scale
physical_scale

Ne jamais mélanger :

display_geometry
physical_state

Ne jamais mélanger :

candidate_law
established_law

Ne jamais mélanger :

conceptual_dimension
Hilbert-space dimension
qubit count

Ne jamais mélanger :

visual 4D projection
literal four-spatial-dimensional physical claim

=====
PARTIE Q – FIN DE TÂCHE
=====

À la fin :

1. exécute tous les tests ;
2. fournis la liste exacte des fichiers modifiés ;
3. indique la version Qiskit avant/après ;
4. indique les API Qiskit migrées ;
5. montre l'architecture finale ;
6. indique les limitations restantes ;
7. crée un document :

QUANTUM_FRACTALOS_SPATIAL.md

8. crée un document :

COHERENCE_ENGINE.md

9. crée un document :

QISKIT_MIGRATION.md

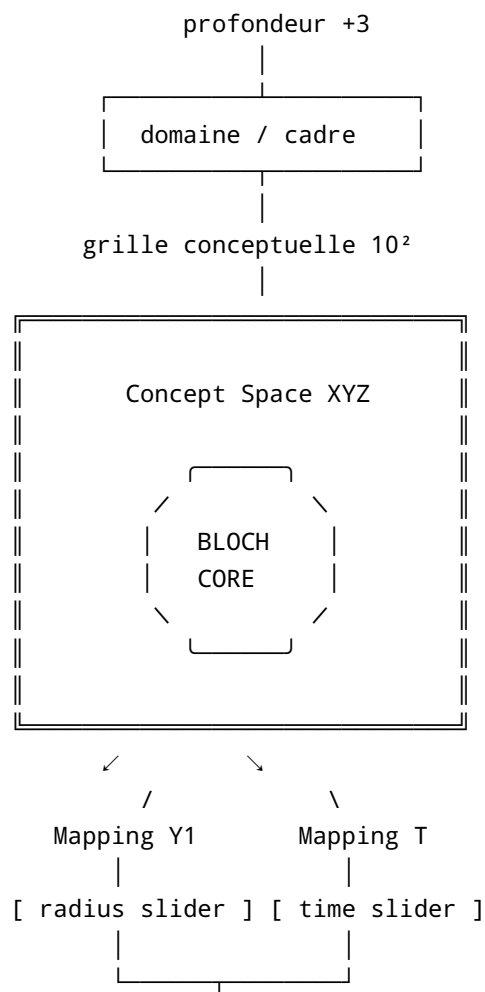
10. ne proclame aucune nouvelle loi physique.

Commence maintenant par l'audit du repository, puis implémente le MVP.

Le choix d'un moteur Web interactif tel que Three.js est cohérent avec les besoins du MVP : la bibliothèque possède des contrôles d'orbite/pan/zoom et des mécanismes de sélection d'objets, tandis que les visualisations natives de Qiskit restent particulièrement adaptées comme sorties scientifiques de référence côté Python. ¹⁸

Critères scientifiques et visuels à atteindre aujourd'hui

Je viserais un écran ressemblant conceptuellement à ceci :



▼
geometry/state

Puis un inspecteur latéral :

VISIBLE SPACE

XYZ
dimension: 3
depth: 0
scale: 10^0

HIDDEN / CONTROL SPACES

Y1 dimension 1
T dimension 1
P dimension 1

ACTIVE MAPPINGS

Y1 -> sphere.radius
T -> sphere.position
P -> quantum.ry.angle

QUANTUM STATE

<X>
<Y>
<Z>
purity
selected qubit

COHERENCE

shared invariant
transform equivalence
dimensional consistency

La démonstration cinq dimensions la plus simple

Prends :

$$\mathcal{S} = (X, Y, Z, T, \theta).$$

La vue affiche :

$$(X, Y, Z)$$

et possède deux contrôles :

$$T$$

et :

$$\theta.$$

Le temps modifie la position :

$$\mathbf{r} = \mathbf{r}(T).$$

Et θ pilote par exemple un paramètre d'un circuit :

$$R_y(\theta).$$

Qiskit recalcule alors :

$$|\psi(\theta)\rangle$$

puis :

$$\langle X \rangle, \langle Y \rangle, \langle Z \rangle.$$

Ces valeurs déplacent le vecteur de la sphère de Bloch.

Tu obtiens donc une chaîne réellement fonctionnelle :

$$\text{espace de paramètres} \rightarrow \text{fonction} \rightarrow \text{circuit} \rightarrow \text{état} \rightarrow \text{observable} \rightarrow \text{géométrie}$$

C'est précisément le genre d'architecture auquel les circuits paramétrés et les primitives V2 vectorisées de Qiskit se prêtent bien. ¹⁹

Ce que devient la notion d'« espace »

Le gain architectural essentiel est que **XYZ cesse d'être le seul espace privilégié**.

On obtient plutôt :

$$\mathcal{S}_{\text{visible}} \times \mathcal{S}_{\text{controls}} \times \mathcal{S}_{\text{state}} \times \mathcal{S}_{\text{laws}}$$

avec des applications entre eux.

Par exemple :

$$\theta \in \mathcal{S}_{\text{parameter}}$$

agit sur :

$$U(\theta) \in \mathcal{S}_{\text{operators}},$$

qui agit sur :

$$|\psi\rangle \in \mathcal{S}_{\text{quantum}},$$

qui produit :

$$(\langle X \rangle, \langle Y \rangle, \langle Z \rangle)$$

qui apparaît enfin dans :

$$\mathcal{S}_{\text{visual}}.$$

Cette séparation rend possible quelque chose de beaucoup plus vaste que « visualiser un circuit » : **Fractalos devient un moteur de composition de représentations scientifiques.**

Ce que je figerais après `campaign_0011`

Je ne modifierais pas silencieusement le noyau expérimental ayant produit H01/H02.

Je créerais clairement une nouvelle génération :

Quantum Forge V0.9 / Fractalos Spatial

et je garderais un mode de compatibilité V0.8.

La campagne `0011` devient alors un **baseline de non-régression** :

toute modernisation Qiskit doit pouvoir démontrer que les expériences historiques importantes conservent leurs résultats dans les tolérances pré-enregistrées.

C'est particulièrement important parce que Qiskit évolue activement : sa version 2.5 apporte un nouveau pipeline de compilation multi-IR naissant, de nouvelles optimisations de transpilation et diverses modifications internes ; une migration de dépendance ne doit donc jamais être confondue avec une nouvelle preuve scientifique. ¹⁴

Direction à moyen terme

Le véritable potentiel de **Quantum Fractalos OS Spatial** n'est pas, à mon sens, de transformer les lois de la science en jolies sphères.

C'est de construire un système dans lequel une loi possède simultanément :

forme symbolique,
domaine de validité,
unités,
paramètres,
géométrie,
dynamique,
invariants,
relations avec d'autres lois.

Puis de pouvoir « zoomer » conceptuellement.

À un niveau :

formule individuelle.

Au-dessus :

famille de formules.

Au-dessus :

cadre théorique.

Au-dessus :

domaine scientifique.

Et entre eux :

mappings + invariants + transformations + observations

plutôt que :

anomalies seules .

La régression symbolique et l'identification de systèmes peuvent devenir des outils de proposition à l'intérieur de cet espace : AI Feynman montre l'intérêt d'exploiter structure, symétries et séparabilité, et SINDy montre qu'une dynamique peut parfois être retrouvée sous forme parcimonieuse à partir de trajectoires. ¹⁷

Et pour le quantum, le centre demeure mathématiquement propre :

Bloch sphere pour le qubit

puis, quand la complexité augmente :

état quantique complet hors de la simple sphère

avec projection vers les observables utiles. Le formalisme de Bloch généralisé rappelle précisément que les espaces d'états de dimension supérieure n'entrent pas fidèlement dans une simple représentation tridimensionnelle. ⁴

Le résultat visé peut donc être résumé ainsi :

Quantum Fractalos OS Spatial

=

moteur scientifique multi-espace + géométrie récursive + Qiskit moderne + cohérence inter-domaines

avec la règle fondamentale :

la visualisation peut être audacieuse ; la signification scientifique doit rester stricte.

- 1 Sparse identification of nonlinear dynamical systems
https://arxiv.org/abs/1509.03580?utm_source=chatgpt.com
- 2 Enhancing Symbolic Regression and Universal Physics ...
https://arxiv.org/html/2411.15919v1?utm_source=chatgpt.com
- 3 qiskit.visualization.plot_bloch_multivector
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.visualization.plot_bloch_multivector?utm_source=chatgpt.com
- 4 [0806.1174] Bloch vectors for qudits - Quantum Physics
https://arxiv.org/abs/0806.1174?utm_source=chatgpt.com
- 5 qiskit.visualization.plot_state_qsphere
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.visualization.plot_state_qsphere?utm_source=chatgpt.com
- 6 Qiskit SDK release notes | IBM Quantum Documentation
<https://quantum.cloud.ibm.com/docs/api/qiskit/release-notes>
- 7 primitives (latest version) | IBM Quantum Documentation
https://quantum.cloud.ibm.com/docs/api/qiskit/primitives?utm_source=chatgpt.com
- 8 StatevectorSampler (latest version)
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.primitives.StatevectorSampler?utm_source=chatgpt.com
- 9 19 StatevectorEstimator (latest version)
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.primitives.StatevectorEstimator?utm_source=chatgpt.com
- 10 Statevector (latest version) | IBM Quantum Documentation
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.quantum_info.Statevector?utm_source=chatgpt.com
- 11 DensityMatrix (latest version) | IBM Quantum Documentation
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.quantum_info.DensityMatrix?utm_source=chatgpt.com
- 12 SparseObservable (latest version)
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.quantum_info.SparseObservable?utm_source=chatgpt.com
- 13 qpy (latest version) | IBM Quantum Documentation
https://quantum.cloud.ibm.com/docs/api/qiskit/qpy?utm_source=chatgpt.com
- 14 Qiskit SDK 2.5 release notes | IBM Quantum Documentation
<https://quantum.cloud.ibm.com/docs/en/api/qiskit/release-notes/2.5>
- 15 18 OrbitControls – three.js docs
https://threejs.org/docs/pages/OrbitControls.html?utm_source=chatgpt.com
- 16 Discovering governing equations from data by sparse ...
https://www.pnas.org/doi/10.1073/pnas.1517384113?utm_source=chatgpt.com
- 17 AI Feynman: a Physics-Inspired Method for Symbolic Regression
https://arxiv.org/abs/1905.11481?utm_source=chatgpt.com